

Mathematical Preliminaries

Nagesh

May 2026

Everything is "Human Claims".

Search your mind, or pay attention to the conversations you have with other people, and you will discover that there are no real boundaries between science and any other discipline that attempts to make valid claims about the world on the basis of evidence and logic. When such claims and their methods of verification admit of experiment and/or mathematical description, we tend to say that our concerns are "Scientific"; when they relate to matters more abstract, or to the consistency of our thinking itself, we often say that we are being "Philosophical"; when we merely want to know how people behaved in the past, we dub our interests "historical" or "journalistic"; and when a person's commitment to evidence and logic grows dangerously thin or simply snaps under the burden of fear, wishful thinking, tribalism, or ecstasy, we recognize that he is being "Religious."

— Sam Harris (Waking Up , page number 199)

Contents

Functional Approximation: A Brief Overview	3
The Problem of the Deterministic Worldview	4
Probability Theory: A Beginner Introduction	4
Probability Measure	6
Probability Triplet	9
Idea of a Random Variable	10
Effect of Random Variables	14
Joint Distribution	18
Conditional Distribution	19

Marginal Distribution	20
Notes	21
IID Assumption on Datasets	23
Notes	28
Distribution Estimation	30
Fundamental Problem of Machine Learning	30
Distribution Estimation Examples	31
Important Note	33
Probability Density Function	34
Notes on Probability Density Functions	36
Basics of Calculus	38
Basics of Linear Algebra	40
Basics of Optimization	42
Variants of Gradient Descent	44

Mathematical Preliminaries

We don't need to know how an engine works if you just need to drive your bike.....

BUT ...

You need to know the inner workings to build an efficient one on your own, for your own. It is for the second reason we are studying mathematics here.

Functional Approximation: A Brief Overview

Consider a dataset consisting of input-output pairs:

X : Domain (Input Space)

Y : Range (Output Space)

Suppose we are given a collection of observations

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\},$$

where each pair (x_i, y_i) represents an observed input and its corresponding output.

We assume that there exists an underlying function

$$f : X \rightarrow Y,$$

which governs the relationship between inputs and outputs. However, this function is unknown to us.

The goal of **functional approximation** is therefore:

Given the dataset D , estimate the unknown function f .

This is one of the fundamental problems in science, engineering, and machine learning.

How do we approximate an unknown function?

Since the true function f is not directly accessible, we begin with an initial guess of the function. Each such guess corresponds to a **model**.

Let the model be denoted by

$$f_\theta,$$

where θ represents a set of adjustable parameters.

The objective is to refine these parameters so that the model predictions become increasingly consistent with the observed data. This refinement is performed using an optimization algorithm, which minimizes a suitable error (or loss) function:

$$\mathcal{L}(\theta) = \sum_{i=1}^N \ell(f_\theta(x_i), y_i),$$

where $\ell(\cdot, \cdot)$ measures the discrepancy between the predicted and observed outputs.

Thus, functional approximation can be viewed as the process of iteratively improving a parameterized model using data, until the model provides a satisfactory approximation of the unknown function f .

The Problem of the Deterministic Worldview

For many physical systems, measurable quantities such as force, velocity, acceleration, and mass allow us to construct precise deterministic relationships. In such settings, the behavior of the system can often be described using well-defined mathematical laws.

However, in many real-world problems, especially those involving biological, social, or cognitive systems, the quantities of interest are often abstract, noisy, or not rigorously defined. In these cases, a purely deterministic worldview becomes insufficient.

For example, the notion of a *label* in machine learning is often human-centric and synthetic. Labels such as “cat”, “spam”, “good”, or “relevant” are not fundamental physical quantities; rather, they are abstractions imposed by human interpretation.

As a result, the underlying function

$$f : X \rightarrow Y$$

becomes highly complex and difficult to derive directly from first-principle physics.

Instead of deriving f analytically, we resort to **statistical methods**. The central idea is to make repeated observations of the system and use the collected data to estimate the unknown functional relationship.

Given a dataset

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\},$$

we attempt to infer a function f that captures the statistical regularities present in the observations.

Thus, modern functional approximation is fundamentally statistical in nature:

$$\text{Repeated observations} \implies \text{Estimate of } f.$$

Rather than seeking exact deterministic laws, we seek models that approximate the observed behavior with high predictive accuracy.

Probability Theory: A Beginner Introduction

One of the most subtle ideas in probability theory is randomness itself. Strictly speaking, randomness is not defined with absolute precision at an intuitive level; rather, it is assumed as a primitive concept upon which the mathematical framework of probability is constructed.

Sample Space and Events

Let

$$\Omega$$

denote the **sample space**, i.e., the set of all possible outcomes of a random experiment.

An **event** is a subset of the sample space. A **trial** is a single realization or occurrence of the experiment.

Example: Tossing a Coin

Consider the experiment of tossing a coin once.

The sample space is

$$\Omega = \{\text{Head}, \text{Tail}\}.$$

Possible events include:

$$A = \{\text{Head}\},$$

$$B = \{\text{Tail}\},$$

or even

$$C = \{\text{Head}, \text{Tail}\} = \Omega.$$

A single toss producing “Head” is one trial (or realization).

Important Note: We do not attempt to rigorously define what a “random experiment” is at this stage. Instead, we assume that such experiments exist and that repeated observations generate outcomes belonging to a sample space.

Assigning Measures

Once a sample space is defined, probability theory assigns a **measure** to it.

The notion of a measure is already familiar from geometry and physics:

- For a one-dimensional object, the measure is its **length**.
- For a two-dimensional object, the measure is its **area**.
- For a three-dimensional object, the measure is its **volume**.
- For an n -dimensional object, the measure is its **hypervolume**.

Examples of Measures

- The interval

$$[0, 2]$$

has length

$$2.$$

- A square of side length 3 has area

$$3 \times 3 = 9.$$

- A cube of side length 2 has volume

$$2^3 = 8.$$

Probability theory generalizes this idea of measurement to collections of outcomes in a sample space.

NOTE : Abstract Sample Spaces

A sample space need not correspond to a physical geometric object. It can also be an abstract collection.

For example, consider the set of all English words:

$$\Omega = \{\text{all valid English words}\}.$$

This sample space has no obvious geometric length, area, or volume, yet probability measures can still be assigned to subsets of it. For instance, one may define probabilities based on how frequently words appear in a language corpus.

Probability Measure

Consider a sample space

$$\Omega,$$

which contains all possible outcomes of a random experiment.

Now consider the collection of subsets of the sample space. Let

$$\mathcal{F}$$

denote a collection of subsets of Ω .

The elements of $\mathcal{F}$ are called **events**.

The primary objective of probability theory is to assign a meaningful **measure** to these events.

Probability as a Measure

A probability measure is a special type of measure defined as

$$P : \mathcal{F} \rightarrow [0, 1].$$

That is, to every event $A \in \mathcal{F}$, the probability measure assigns a real number between 0 and 1.

$$A \mapsto P(A)$$

The quantity $P(A)$ represents the **likelihood of occurrence** of the event A .

Properties of Probability Measure

The probability measure behaves similarly to familiar geometric measures such as length, area, and volume.

For events $A, B \in \mathcal{F}$, the probability measure satisfies the following properties:

1. **Non-negativity**

$$P(A) \geq 0, \quad \forall A \in \mathcal{F}.$$

2. **Normalization**

$$P(\Omega) = 1, \quad P(\emptyset) = 0.$$

Here,

Ω

denotes the entire sample space, and

$\emptyset$

denotes the empty set.

3. **Additivity**

If

$$A \cap B = \emptyset,$$

i.e., A and B are disjoint events, then

$$P(A \cup B) = P(A) + P(B).$$

Interpretation

Probability theory can therefore be viewed as a framework for assigning numerical measures to events in a sample space.

Just as geometric measure quantifies size, probability measure quantifies the **likelihood of occurrence** of events.

Understanding the Properties of Probability Measure with Examples

Consider the experiment of tossing a fair coin once.

The sample space is

$$\Omega = \{\text{Head}, \text{Tail}\}.$$

Let

$$A = \{\text{Head}\}, \quad B = \{\text{Tail}\}.$$

Since the coin is fair,

$$P(A) = \frac{1}{2}, \quad P(B) = \frac{1}{2}.$$

(i) Non-negativity

For every event $A \in \mathcal{F}$,

$$P(A) \geq 0.$$

Interpretation: Probability can never be negative. A negative likelihood has no physical meaning.

Example:

For the coin toss,

$$P(\{\text{Head}\}) = \frac{1}{2} \geq 0,$$

and

$$P(\{\text{Tail}\}) = \frac{1}{2} \geq 0.$$

Even impossible events satisfy this:

$$P(\emptyset) = 0.$$

Thus, all probabilities are non-negative numbers.

(ii) Normalization

$$P(\Omega) = 1, \quad P(\emptyset) = 0.$$

Interpretation:

- The probability that *something* in the sample space occurs is always 1.
- The probability of an impossible event is 0.

Example:

For the coin toss,

$$\Omega = \{\text{Head}, \text{Tail}\}.$$

One of these outcomes must occur after tossing the coin. Hence,

$$P(\Omega) = P(\{\text{Head}, \text{Tail}\}) = 1.$$

Now consider the empty event:

$$\emptyset.$$

This event contains no outcomes and therefore can never occur:

$$P(\emptyset) = 0.$$

(iii) Additivity

If two events are disjoint,

$$A \cap B = \emptyset,$$

then

$$P(A \cup B) = P(A) + P(B).$$

Interpretation:

If two events cannot occur simultaneously, then the probability that either event occurs is simply the sum of their individual probabilities.

Example:

For the coin toss,

$$A = \{\text{Head}\}, \quad B = \{\text{Tail}\}.$$

Clearly,

$$A \cap B = \emptyset,$$

because a single toss cannot produce both Head and Tail simultaneously.

Now,

$$A \cup B = \{\text{Head}, \text{Tail}\} = \Omega.$$

Hence,

$$P(A \cup B) = P(A) + P(B) = \frac{1}{2} + \frac{1}{2} = 1.$$

This agrees with the fact that either Head or Tail must occur.

Summary

The three properties ensure that probability behaves consistently as a mathematical measure of likelihood:

- Non-negativity prevents meaningless negative probabilities.
- Normalization ensures total certainty over the sample space.
- Additivity allows probabilities of disjoint events to combine naturally.

Probability Triplet

A probability space is completely described by the triplet

$$(\Omega, \mathcal{F}, P),$$

where

- Ω is the **sample space**,
- $\mathcal{F}$ is the collection of events,
- P is the probability measure.

Together, these three components define the mathematical structure of probability theory.

Idea of a Random Variable

Once a probability space is defined, we introduce the concept of a **random variable**.

A random variable is a function

$$X : \Omega \rightarrow \mathbb{R}^d.$$

That is, it maps outcomes from the abstract sample space into a numerical space.

$$\text{Random Variable: Abstract Space} \longrightarrow \text{Numerical Space}$$

Important Note:

Despite its name, a random variable is neither truly “random” nor necessarily a “variable” in the usual sense. Mathematically, it is simply a deterministic function acting on the sample space.

The randomness originates from the outcome

$$\omega \in \Omega,$$

not from the mapping itself.

Why Do We Need Random Variables?

Sample spaces are often abstract and difficult to work with directly. Random variables allow us to translate these abstract outcomes into numbers, vectors, or tensors where standard mathematical tools can be applied.

In essence, random variables convert qualitative or symbolic information into quantitative representations.

Example: Tossing a Coin

Consider the sample space

$$\Omega = \{\text{Head}, \text{Tail}\}.$$

Define a random variable

$$X : \Omega \rightarrow \mathbb{R}$$

as

$$X(\text{Head}) = 1, \quad X(\text{Tail}) = 0.$$

Here, the abstract outcomes “Head” and “Tail” are mapped into numbers.

Thus,

$$\{\text{Head}, \text{Tail}\} \longrightarrow \{1, 0\}.$$

Now numerical operations such as averages, variances, and expectations become possible.

Example: Images as Random Variables

Consider a dataset of images.

The sample space may consist of abstract objects:

$$\Omega = \{\text{all possible images}\}.$$

Directly working with abstract image objects is difficult mathematically.

A random variable can map each image into a numerical tensor:

$$X(\omega) = \text{pixel intensity matrix}.$$

For example,

$$X : \Omega \rightarrow \mathbb{R}^{H \times W \times 3},$$

where

- H = image height,
- W = image width,
- 3 corresponds to RGB color channels.

Thus, an abstract image becomes a numerical object that can be processed using linear algebra, optimization, and machine learning methods.

Therefore, random variables serve as the bridge between abstract probabilistic outcomes and numerical mathematics.

The Complete Picture

The complete probabilistic framework can now be summarized as follows:

$$X : \Omega \rightarrow \mathbb{R}^d$$

where

- Ω is the sample space associated with an underlying random experiment,
- X is the random variable,
- $\mathbb{R}^d$ is the numerical representation space.

The sample space

$$\Omega$$

contains abstract outcomes generated by the underlying random experiment.

The random variable

$$X$$

acts as a deterministic mapping that converts these abstract outcomes into numerical objects.

Thus, the randomness belongs to the sample space and the experiment itself, while the random variable is simply a mathematical function.

Example: Images

Consider the case of images.

An image can be represented numerically as a vector or tensor in a high-dimensional space:

$$\mathbb{R}^d.$$

For example, a colored image of size

$$H \times W \times 3$$

can be flattened into a vector:

$$x \in \mathbb{R}^d, \quad d = H \times W \times 3.$$

Hence, an image is ultimately an element of a numerical vector space.

However, this numerical object originates from an abstract outcome in the sample space:

$$\omega \in \Omega.$$

The role of the random variable is therefore:

$$X(\omega) = x,$$

where

$$x \in \mathbb{R}^d$$

is the numerical representation of the outcome.

Conceptually,

$$\text{Underlying Random Experiment} \implies \Omega \xrightarrow{X} \mathbb{R}^d$$

Thus:

- The sample space describes the abstract probabilistic world.
- The random variable translates this world into numbers.
- Mathematics, statistics, and machine learning operate in this numerical space.

Deterministic vs Probabilistic View of Data

There are two fundamentally different ways to think about a datapoint.

Deterministic View

In the deterministic viewpoint, a datapoint is simply treated as a vector in a numerical space:

$$x \in \mathbb{R}^d.$$

For example, an image is viewed merely as a collection of pixel values:

$$x = (x_1, x_2, \dots, x_d).$$

In this perspective, the datapoint is just a geometric object in a high-dimensional vector space.

Probabilistic View

In the probabilistic viewpoint, the datapoint is not considered an isolated vector.

Instead, it is viewed as the output of a random variable:

$$X : \Omega \rightarrow \mathbb{R}^d.$$

Thus, a datapoint

$$x \in \mathbb{R}^d$$

is interpreted as

$$x = X(\omega),$$

for some underlying outcome

$$\omega \in \Omega.$$

Here,

- Ω represents the hidden sample space associated with an underlying random experiment,
- X is the random variable,
- $\mathbb{R}^d$ is the numerical representation space.

Therefore, from the probabilistic perspective:

$$\text{Datapoint} \neq \text{just a vector}$$

Rather,

$$\text{Datapoint} = \text{realization of a random variable.}$$

Importantly, we usually do not have direct access to the underlying sample space

$$\Omega.$$

We only observe the numerical realizations produced by the random variable:

$$x_1, x_2, \dots, x_N \in \mathbb{R}^d.$$

This viewpoint is central to statistics and machine learning, where observed data is assumed to arise from an underlying probabilistic generative process.

Effect of Random Variables

A random variable is a mapping

$$X : \Omega \rightarrow \mathbb{R}^d.$$

Naturally, one may ask:

What happens to the subsets of the sample space under this mapping?

The answer is that the random variable transfers the probabilistic structure of the sample space onto the numerical space.

From Sample Space to Numerical Space

Recall the probability triplet:

$$(\Omega, \mathcal{F}, P).$$

Here,

- Ω : sample space,
- $\mathcal{F}$: collection of measurable events,
- P : probability measure on $\mathcal{F}$.

Now consider a random variable

$$X : \Omega \rightarrow \mathbb{R}^d.$$

Under this mapping:

$$\Omega \implies \mathbb{R}^d$$

The subsets (events) in

$$\mathcal{F}$$

are transformed into measurable subsets of

$$\mathbb{R}^d.$$

These measurable subsets form the **Borel sigma algebra**, denoted by

$$\mathcal{B}(\mathbb{R}^d).$$

Thus,

$$\mathcal{F} \implies \mathcal{B}(\mathbb{R}^d).$$

Similarly, the probability measure P induces a probability distribution on the numerical space:

$$P \implies P_X.$$

Here,

$$P_X$$

is called the **distribution of the random variable X** .

Therefore, the complete transformation is:

$$\Omega \xrightarrow{X} \mathbb{R}^d$$

$$\mathcal{F} \xrightarrow{X} \mathcal{B}(\mathbb{R}^d)$$

$$P \xrightarrow{X} P_X$$

Interpretation

The random variable does not merely assign numbers to outcomes. It transports the entire probabilistic structure from the abstract sample space into a numerical space where analysis becomes possible.

As a result:

- abstract events become measurable geometric regions,
- probabilities become probability distributions,
- probability theory becomes analyzable using calculus, geometry, and linear algebra.

Distribution as Probability of Events

A random variable

$$X : \Omega \rightarrow \mathbb{R}^d$$

maps abstract outcomes into a numerical space.

Now consider a subset

$$B \subseteq \mathbb{R}^d.$$

A natural question arises:

What is the probability that the random variable takes values inside the set B ?

The answer is given by the probability distribution induced by the random variable.

Preimage of a Set

The subset

$$B \subseteq \mathbb{R}^d$$

corresponds to a subset of the sample space:

$$X^{-1}(B) = \{\omega \in \Omega : X(\omega) \in B\}.$$

This set contains all outcomes in the sample space whose numerical representations fall inside B .

The probability distribution evaluates:

$$P_X(B) = P(X^{-1}(B)).$$

Thus, the probability distribution of the random variable is fundamentally the probability of the corresponding subset in the underlying sample space.

Conceptually,

$$\text{Subset in } \mathbb{R}^d \iff \text{Subset in } \Omega$$

and

$$P_X(B) = P(\omega \text{ such that } X(\omega) \in B).$$

Example

Suppose a coin toss experiment has sample space

$$\Omega = \{\text{Head}, \text{Tail}\}.$$

Define the random variable

$$X(\text{Head}) = 1, \quad X(\text{Tail}) = 0.$$

Now consider the subset

$$B = \{1\} \subseteq \mathbb{R}.$$

The corresponding subset in the sample space is

$$X^{-1}(B) = \{\text{Head}\}.$$

Therefore,

$$P_X(B) = P(\text{Head}).$$

If the coin is fair,

$$P_X(\{1\}) = \frac{1}{2}.$$

Thus, the distribution function evaluates the probability of the underlying event corresponding to that numerical region.

Hence, probability distributions are simply probability measures transported from the abstract sample space into numerical space through the random variable.

Important Notes

1. **A distribution function is a measure, not merely a probability value.**

The distribution induced by a random variable is itself a probability measure defined on subsets of the numerical space.

Formally,

$$P_X : \mathcal{B}(\mathbb{R}^d) \rightarrow [0, 1].$$

Thus, the distribution does not evaluate a single point alone; it assigns measures to measurable subsets of the range space.

For a subset

$$B \subseteq \mathbb{R}^d,$$

the distribution evaluates

$$P_X(B).$$

Hence, a distribution is fundamentally a measure-theoretic object.

In continuous spaces, individual points often have zero probability:

$$P_X(\{x\}) = 0,$$

while regions can have nonzero probability:

$$P_X(B) > 0.$$

Therefore, distributions are naturally understood as measures over sets rather than probabilities assigned to isolated points.

2. **A vector-valued random variable can be viewed as multiple correlated scalar random variables.**

Consider a random variable

$$X : \Omega \rightarrow \mathbb{R}^d.$$

This can be decomposed into components:

$$X(\omega) = (X_1(\omega), X_2(\omega), \dots, X_d(\omega)),$$

where each

$$X_i : \Omega \rightarrow \mathbb{R}$$

is a scalar random variable.

Thus, a vector-valued random variable is essentially a collection of scalar random variables defined on the same sample space.

Importantly, these scalar random variables are usually *correlated* because they originate from the same underlying outcome

$$\omega \in \Omega.$$

Example: Images

Consider an RGB image represented as

$$X \in \mathbb{R}^{H \times W \times 3}.$$

Each pixel intensity can be interpreted as a scalar random variable:

$$X_{ijc} : \Omega \rightarrow \mathbb{R},$$

where

- i, j denote spatial location,
- c denotes the color channel.

The complete image is therefore a very high-dimensional correlated random variable.

Neighboring pixels are highly dependent, which gives rise to spatial structure and patterns in images.

Joint Distribution

Consider two random variables

$$X : \Omega \rightarrow \mathbb{R}, \quad Y : \Omega \rightarrow \mathbb{R}.$$

Together, they define a vector-valued random variable:

$$(X, Y) : \Omega \rightarrow \mathbb{R}^2.$$

The **joint distribution** describes the probability measure associated with the pair (X, Y) .

For measurable subsets

$$A, B \subseteq \mathbb{R},$$

the joint distribution evaluates:

$$P(X \in A, Y \in B).$$

Thus, the joint distribution measures the probability that both random variables simultaneously fall inside specified regions.

Example

Consider tossing two fair coins.

Define random variables:

$$X = \begin{cases} 1 & \text{if first coin is Head} \\ 0 & \text{if first coin is Tail} \end{cases}$$

$$Y = \begin{cases} 1 & \text{if second coin is Head} \\ 0 & \text{if second coin is Tail} \end{cases}$$

The sample space is

$$\Omega = \{(H, H), (H, T), (T, H), (T, T)\}.$$

Now,

$$P(X = 1, Y = 1) = P((H, H)) = \frac{1}{4}.$$

Similarly,

$$P(X = 0, Y = 1) = \frac{1}{4}.$$

The complete collection of such probabilities forms the joint distribution.

For continuous variables, the joint distribution is often represented using a joint density:

$$p(x, y).$$

Then probabilities are obtained by integrating over regions:

$$P((X, Y) \in R) = \int_R p(x, y) dx dy.$$

Conditional Distribution

The **conditional distribution** describes the behavior of one random variable when another random variable is known.

Formally,

$$P(Y \in B \mid X \in A).$$

It answers questions of the form:

Given information about X , how does the probability distribution of Y change?

For discrete variables,

$$P(Y = y \mid X = x) = \frac{P(X = x, Y = y)}{P(X = x)},$$

provided

$$P(X = x) > 0.$$

Example

Consider again two fair coin tosses.

Suppose:

X = result of first coin,

Y = result of second coin.

Now ask:

$$P(Y = 1 \mid X = 1).$$

Interpretation:

What is the probability that the second coin is Head given that the first coin is already known to be Head?

Since the coins are independent,

$$P(Y = 1 \mid X = 1) = \frac{1}{2}.$$

Now consider a dependent example.

Suppose:

$$Y = X.$$

Then,

$$P(Y = 1 \mid X = 1) = 1.$$

Thus, conditional distributions capture dependencies between random variables.

Marginal Distribution

The **marginal distribution** describes the probability distribution of a subset of variables while ignoring the remaining variables.

Starting from a joint distribution:

$$P(X, Y),$$

the marginal distribution of X is obtained by summing or integrating over all possible values of Y .

For discrete variables:

$$P(X = x) = \sum_y P(X = x, Y = y).$$

For continuous variables:

$$p_X(x) = \int p(x, y) dy.$$

Thus, marginalization removes unwanted variables.

Example

Consider the joint probabilities:

$$P(X = 0, Y = 0) = \frac{1}{4},$$

$$P(X = 0, Y = 1) = \frac{1}{4},$$

$$P(X = 1, Y = 0) = \frac{1}{4},$$

$$P(X = 1, Y = 1) = \frac{1}{4}.$$

To obtain the marginal distribution of X :

$$P(X = 0) = P(X = 0, Y = 0) + P(X = 0, Y = 1).$$

Hence,

$$P(X = 0) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}.$$

Similarly,

$$P(X = 1) = \frac{1}{2}.$$

Thus, the marginal distribution focuses only on one variable by averaging over the others.

Conceptually:

$$\text{Joint Distribution} \implies \text{Marginal Distribution}$$

by integrating or summing out unnecessary variables.

Notes

1. In general, a datapoint is a d -dimensional vector.

Most modern datasets can be represented as elements of a high-dimensional numerical space:

$$x \in \mathbb{R}^d.$$

Examples include:

- Images $\rightarrow$ pixel vectors,
- Audio $\rightarrow$ waveform vectors,
- Text $\rightarrow$ embedding vectors,
- Protein sequences $\rightarrow$ representation embeddings.

2. **Every datapoint is an element in the range space of a random variable.**

From the probabilistic viewpoint, observed data is not treated as isolated vectors.

Instead,

$$x = X(\omega),$$

where

$$X : \Omega \rightarrow \mathbb{R}^d$$

is a random variable defined on an underlying sample space.

Thus, datapoints are realizations of an underlying probabilistic process.

3. **The distribution function quantifies the likelihood of observing datapoints under X .**

The probability distribution induced by the random variable assigns measures to regions of the numerical space:

$$P_X(B), \quad B \subseteq \mathbb{R}^d.$$

This quantifies how likely the random variable is to generate datapoints inside the region B .

In continuous spaces, the probability of observing an exact point is often zero:

$$P_X(\{x\}) = 0.$$

Therefore, distributions fundamentally measure likelihood over neighborhoods or regions rather than isolated points.

4. **If the distribution is known, the observable probabilistic behavior is completely known.**

Once the distribution of a random variable is specified, all measurable probabilistic properties of the observed data become determined.

This includes:

- probabilities of events,
- expectations,
- correlations,
- variances,
- conditional relationships,
- sampling behavior.

In this sense, the distribution encodes all observable statistical information accessible through the random variable.

5. **The distribution function completely characterizes the observable space induced by the random variable.**

Different underlying sample spaces may induce the same observable distribution under a random variable.

Hence, in practice, machine learning and statistics focus primarily on the induced probability distribution in the numerical space:

$$P_X.$$

The explicit structure of the hidden sample space

$$\Omega$$

is usually inaccessible and unnecessary for computation.

6. **A central objective of generative AI is distribution estimation.**

Given observed samples

$$x_1, x_2, \dots, x_N \sim P_{\text{data}},$$

generative models attempt to estimate the underlying data distribution:

$$P_{\text{data}}.$$

Once this distribution is approximated, new samples can be generated:

$$x_{\text{new}} \sim P_{\text{model}}.$$

Thus, generative AI can be viewed fundamentally as the problem of learning and sampling from high-dimensional probability distributions.

IID Assumption on Datasets

In probability theory and machine learning, datasets are often categorized into two broad classes:

1. IID data
2. Non-IID data

The IID assumption is one of the foundational assumptions in statistics and learning theory.

(i) IID Data

IID stands for:

Independent and Identically Distributed.

Suppose a dataset consists of samples

$$x_1, x_2, \dots, x_N.$$

The dataset is called IID if:

1. Independent

The occurrence of one sample does not influence another.

Formally,

$$P(x_i, x_j) = P(x_i)P(x_j), \quad i \neq j.$$

Thus, samples do not contain sequential or causal dependence.

2. Identically Distributed

Every sample is generated from the same underlying probability distribution:

$$x_i \sim P_X, \quad \forall i.$$

Hence, all datapoints statistically follow the same generating process.

Therefore, IID data is represented as:

$$x_1, x_2, \dots, x_N \stackrel{\text{iid}}{\sim} P_X.$$

Example: IID Dataset

Consider repeatedly tossing a fair coin.

Define:

$$X_i = \begin{cases} 1 & \text{Head} \\ 0 & \text{Tail} \end{cases}$$

Each toss:

- does not depend on previous tosses,
- follows the same probability distribution.

Hence,

$$X_1, X_2, \dots, X_N$$

are IID random variables.

Another machine learning example:

Suppose images are randomly sampled from a very large image database.

If every image is sampled independently using the same procedure, the dataset is approximately IID.

(ii) Non-IID Data

A dataset is called **non-IID** when either:

- samples are dependent,
- distributions change across samples,
- or both.

Thus, the IID assumptions break down.

Example: Time Series

Consider daily stock prices:

$$x_1, x_2, \dots, x_N.$$

Today's price strongly depends on yesterday's price.

Therefore:

$$P(x_t \mid x_{t-1}) \neq P(x_t).$$

The samples are correlated and hence not independent.

Additionally, the statistical properties may change over time due to market conditions.

Thus, stock market data is non-IID.

Example: Language Data

Consider a sentence:

$$(\text{The, cat, sat, on, the, mat}).$$

Words are highly dependent on previous words.

For example:

$$P(\text{sat} \mid \text{cat})$$

is very different from

$$P(\text{sat}).$$

Therefore, language data is strongly non-IID.

This is why sequence models such as transformers and recurrent neural networks are required.

Why IID Matters

The IID assumption greatly simplifies mathematical analysis.

Many classical statistical results rely on IID assumptions, including:

- Law of Large Numbers,
- Central Limit Theorem,

- Maximum Likelihood Estimation,
- Classical supervised learning theory.

However, real-world data is often non-IID.
Examples include:

- language,
- videos,
- time series,
- biological sequences,
- reinforcement learning trajectories,
- social networks.

Modern machine learning is largely concerned with learning meaningful structure from highly non-IID data.

Converting Non-IID Data into Approximately IID Data

An important idea in modern machine learning is that many non-IID problems can be reformulated into approximately IID learning problems by redefining the notion of a sample.

Non-IID behavior usually arises because observations are dependent across time, space, or sequence position.

For example, in language modeling:

$$x_1, x_2, \dots, x_t, \dots$$

the next token depends strongly on previous tokens:

$$P(x_t \mid x_{t-1}, x_{t-2}, \dots) \neq P(x_t).$$

Thus, individual tokens are clearly non-IID.

Idea of Contextual Samples

Instead of treating a single token as a sample, we redefine the sample structure.

Suppose we choose a context length k .

We now construct samples of the form:

$$\underbrace{(x_{t-k}, \dots, x_{t-1})}_{\text{context}}, x_t.$$

That is,

$$\text{Input} = (x_{t-k}, \dots, x_{t-1}), \quad \text{Target} = x_t.$$

Each training example is now a *context-target pair*.

Under this reformulation, the learning problem becomes:

$$P(x_t \mid x_{t-k}, \dots, x_{t-1}).$$

Now the dataset consists of many such context windows:

$$D = \{(c_1, y_1), (c_2, y_2), \dots, (c_N, y_N)\}.$$

These constructed samples are often treated as approximately IID during training.

Interpretation

The underlying sequence itself remains non-IID.

However, by redefining the sample space appropriately, we create a new representation where optimization algorithms can operate under approximate IID assumptions.

Conceptually:

$$\text{Non-IID Sequential Data} \implies \text{Contextual Window Samples} \implies \text{Approximate IID Training Set}.$$

Example: Language Modeling

Consider the sentence:

The cat sat on the mat

Using a context length of 2, we construct samples:

(The, cat) $\rightarrow$ sat

(cat, sat) $\rightarrow$ on

(sat, on) $\rightarrow$ the

Each context-target pair is now treated as one datapoint.

This idea is fundamental in:

- language modeling,
- transformers,

- autoregressive models,
- Markov models,
- time-series forecasting.

Key Insight

IID assumptions are often not properties of the raw data itself, but properties of how we choose to define the sample space.

By redefining samples appropriately, many structured non-IID problems can be transformed into tractable statistical learning problems.

Notes

1. **IID means independence across datapoints, not across dimensions of a datapoint.**

Consider datapoints

$$x_1, x_2, \dots, x_N \in \mathbb{R}^d.$$

The IID assumption states:

$$x_i \stackrel{\text{iid}}{\sim} P_X.$$

This means different datapoints are assumed to be independent realizations from the same distribution.

However, the components inside a datapoint:

$$x_i = (x_{i1}, x_{i2}, \dots, x_{id})$$

are generally *not* independent.

Example:

In an image, neighboring pixels are highly correlated:

$$P(x_{ij} \mid x_{i,j+1}) \neq P(x_{ij}).$$

Thus:

$$\text{IID assumption} \neq \text{independence of dimensions.}$$

It only refers to independence across samples in the dataset.

2. **“Identically distributed” can be viewed in two equivalent ways.**

Suppose:

$$X_1, X_2, \dots, X_N$$

are IID random variables with distribution P_X .

Then the observed dataset:

$$x_1, x_2, \dots, x_N$$

can be interpreted as:

- one realization of N IID random variables,

$$x_i = X_i(\omega),$$

or equivalently,

- N realizations of the same random variable

$$X.$$

Both viewpoints are mathematically equivalent and commonly used in probability and statistics.

Conceptually:

$$X_1, X_2, \dots, X_N \iff X \text{ sampled repeatedly.}$$

3. What should be treated as IID data is often a modeling and design choice.

The notion of a “sample” is not fixed by nature.

Instead, the practitioner chooses how to define the sample space and datapoints.

Example:

In language modeling:

- treating individual words as datapoints leads to strong non-IID structure,
- treating context-target windows as datapoints creates approximately IID samples.

Similarly:

- a video may be treated frame-wise,
- patch-wise,
- sequence-wise,
- or clip-wise.

Different choices induce different probabilistic structures.

Thus, IID structure is often not an intrinsic property of raw data itself, but a consequence of how we define:

sample space, random variable, and datapoint representation.

Distribution Estimation

Suppose we are given a dataset

$$D = \{(x_i, y_i)\}_{i=1}^N$$

generated independently from an unknown distribution:

$$(x_i, y_i) \stackrel{\text{iid}}{\sim} P_{XY}.$$

Here,

$$x_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R}^k \quad \text{or} \quad y_i \in \{1, 2, \dots, k\}.$$

Typically:

X : input/features/data,

Y : label/output/target.

Both X and Y are random variables defined on an underlying sample space:

$$X : \Omega \rightarrow \mathbb{R}^d,$$

$$Y : \Omega \rightarrow \mathbb{R}^k.$$

The true probability distribution

$$P_{XY}$$

is unknown.

We only observe samples generated from it.

Fundamental Problem of Machine Learning

Given a dataset

$$D \sim \text{unknown distribution},$$

the central goals of machine learning are:

1. Estimate the underlying probability distribution,
2. Generate/sample new datapoints from the estimated distribution.

Conceptually:

Observed Samples $\implies$ Estimate Distribution $\implies$ Generate New Samples.

Thus, machine learning can fundamentally be viewed as a problem of distribution estimation.

Distribution Estimation Examples

(i) Estimating Conditional Distributions

One common objective is estimating:

$$P(Y | X).$$

This corresponds to learning how outputs depend on inputs.

Classification

For classification:

$$Y \in \{1, 2, \dots, k\}.$$

The model estimates:

$$P(Y = c | X = x),$$

which gives the probability that input x belongs to class c .

Example:

Given an image x , estimate:

$$P(\text{cat} | x), \quad P(\text{dog} | x).$$

The predicted label is usually:

$$\arg \max_c P(Y = c | X = x).$$

Regression

For regression:

$$Y \in \mathbb{R}^k.$$

The objective is to estimate a continuous conditional distribution:

$$P(Y | X).$$

Example:

Given:

$$x = \text{house features},$$

predict:

$$y = \text{house price}.$$

Thus:

$$P(Y | X = x)$$

describes possible price values conditioned on the input features.

This entire framework is called:

Supervised Learning.

because labeled pairs

$$(x_i, y_i)$$

are available.

(ii) Estimating Marginal or Joint Distributions

Another major objective is estimating:

$$P_X, \quad P_Y, \quad P_{XY}.$$

Here the goal is not necessarily prediction of labels, but understanding or generating the structure of the data itself.

Estimating P_X

Learn the distribution of datapoints alone.

Example:

Given many face images:

$$x_1, x_2, \dots, x_N,$$

estimate:

$$P_X.$$

Once estimated, we can sample:

$$x_{\text{new}} \sim P_X,$$

to generate new realistic faces.

This is the foundation of:

- GANs,
- VAEs,
- Diffusion models.

Estimating Joint Distributions

Estimate:

$$P(X, Y).$$

Example:

Suppose:

$$\begin{aligned} X &= \text{medical image,} \\ Y &= \text{disease condition.} \end{aligned}$$

Estimating:

$$P(X, Y)$$

captures relationships between images and diseases jointly.

This enables:

- representation learning,
- multimodal generation,
- structured probabilistic modeling.

These approaches are broadly categorized as:

Unsupervised Learning

or

Self-supervised Learning,

depending on how the learning signal is constructed.

Important Note

In practice, directly estimating full probability distributions in high dimensions is extremely difficult.

Therefore, we usually estimate distributions indirectly through functions or summaries of distributions.

Examples include:

- moments:

$$\mathbb{E}[X], \quad \text{Var}(X),$$

- correlations,
- conditional expectations,
- likelihood functions,
- neural network parameterizations.

Thus, modern machine learning often approximates probability distributions using parameterized functions rather than explicit analytical formulas.

Probability Density Function

For continuous random variables, probabilities are described using a **probability density function (PDF)**.

Suppose

$$X : \Omega \rightarrow \mathbb{R}^d$$

is a continuous random variable.

A probability density function is a non-negative function

$$p(x)$$

such that probabilities are obtained by integrating over regions of the space.

Formally, for a region

$$A \subseteq \mathbb{R}^d,$$

the probability that the random variable falls inside A is:

$$P(X \in A) = \int_A p(x) dx.$$

Thus, the PDF itself is not a probability.

Instead, it is a **density** whose integral over a region gives probability.

Properties of a PDF

A valid probability density function satisfies:

1. Non-negativity:

$$p(x) \geq 0.$$

2. Normalization:

$$\int_{\mathbb{R}^d} p(x) dx = 1.$$

The normalization condition ensures that total probability over the entire space equals 1.

Important Interpretation

For continuous random variables:

$$P(X = x) = 0.$$

A single point has zero probability.

Only regions have nonzero probability:

$$P(a \leq X \leq b) = \int_a^b p(x) dx.$$

Thus:

$$\text{PDF} \neq \text{probability}.$$

Instead:

$$\text{Probability} = \text{Area/Volume under the density}.$$

Example: Uniform Distribution

Consider a random variable uniformly distributed between 0 and 1.

The density function is:

$$p(x) = \begin{cases} 1, & 0 \leq x \leq 1, \\ 0, & \text{otherwise.} \end{cases}$$

:contentReference[oaicite:0]index=0

The graph is a constant horizontal line between 0 and 1.

Now compute the probability:

$$P(0.2 \leq X \leq 0.5).$$

This equals the area under the density curve:

$$\begin{aligned} P(0.2 \leq X \leq 0.5) &= \int_{0.2}^{0.5} 1 \, dx. \\ &= 0.5 - 0.2 = 0.3. \end{aligned}$$

Thus, the probability corresponds geometrically to area.

Example: Gaussian Distribution

One of the most important PDFs in probability theory is the Gaussian (Normal) distribution:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Here:

- μ is the mean,
- σ^2 is the variance.

The Gaussian density assigns high density near the mean and lower density farther away.

Many natural phenomena approximately follow Gaussian distributions due to aggregation of many random effects.

Connection to Machine Learning

In machine learning, density functions describe how likely datapoints are under a model.

Examples:

- Classification:

$$P(Y \mid X)$$

- Generative modeling:

$$P_X(x)$$

- Latent variable models:

$$P(X, Z)$$

Modern generative AI models are fundamentally methods for approximating complex high-dimensional probability densities.

Notes on Probability Density Functions

1. **A density function is a mapping from the range space to non-negative real numbers.**

For a random variable

$$X : \Omega \rightarrow \mathbb{R}^d,$$

the associated probability density function is:

$$p_X : \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}.$$

Thus, the density function operates entirely in the numerical range space of the random variable.

It assigns a non-negative density value to every point:

$$x \mapsto p_X(x).$$

2. **Evaluating the density at a point does not give probability.**

For continuous random variables:

$$P(X = x) = 0.$$

Therefore,

$$p_X(x)$$

is *not* the probability of observing exactly x .

Instead, it measures how densely probability mass is concentrated around that point.

Valid probabilities are obtained only after integration over regions:

$$P(X \in A) = \int_A p_X(x) dx.$$

For example:

$$P(a \leq X \leq b) = \int_a^b p_X(x) dx.$$

Thus:

$$\text{Density at a point} \neq \text{Probability at a point.}$$

Probability corresponds to accumulated area (or volume) under the density curve.

3. Most machine learning algorithms operate on density functions rather than abstract distribution measures.

Working directly with abstract probability measures is mathematically difficult.

Density functions provide a more convenient analytical representation because they allow the use of:

- calculus,
- optimization,
- gradients,
- likelihood functions,
- differential equations.

For example, maximum likelihood estimation typically optimizes:

$$p_\theta(x)$$

or equivalently:

$$\log p_\theta(x).$$

Modern generative models such as:

- diffusion models,
- normalizing flows,
- variational autoencoders,

are fundamentally density estimation procedures.

4. **Not all random variables possess density functions.**

Density functions exist only for absolutely continuous distributions.

Discrete random variables instead use probability mass functions (PMFs).

Example:

$$P(X = 0) = 0.5, \quad P(X = 1) = 0.5.$$

Here probabilities are assigned directly to points.

Some distributions are even more complicated and possess neither a standard density nor a simple discrete mass function.

Example:

- mixed distributions,
- singular distributions,
- fractal-like measures.

Thus, probability density functions are extremely useful, but they are not universally applicable to all random variables.

Basics of Calculus

Calculus is the mathematics of **change** and **accumulation**. It provides tools to study how quantities vary and how local changes influence global behavior.

The two central ideas in calculus are:

1. Differentiation
2. Integration

Differentiation

A derivative measures the rate of change of a function.

Given a function

$$f : \mathbb{R} \rightarrow \mathbb{R},$$

the derivative of f at x is defined as

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Geometrically:

- derivative = slope of the tangent,
- derivative measures local sensitivity,

- derivative tells how fast a quantity changes.

Example

$$f(x) = x^2$$

Then:

$$f'(x) = 2x.$$

At $x = 3$:

$$f'(3) = 6.$$

This means the function changes approximately by 6 units for a small unit change near $x = 3$.

Partial Derivatives

For multivariable functions:

$$f(x_1, x_2, \dots, x_d),$$

we differentiate with respect to one variable while keeping others fixed.

Example:

$$f(x, y) = x^2 + y^2.$$

Then:

$$\frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 2y.$$

Gradient

The gradient combines all partial derivatives into a vector:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_d} \right).$$

The gradient points in the direction of maximum increase of the function.

Integration

Integration measures accumulation.

For a continuous function:

$$f(x),$$

the integral over an interval is:

$$\int_a^b f(x) dx.$$

Geometrically, this represents area under the curve.

Example

$$f(x) = x.$$

Then:

$$\int_0^1 x \, dx = \frac{1}{2}.$$

Thus, the area under the line from 0 to 1 equals 0.5.

Integration is fundamental in probability theory because probabilities are computed using integrals over density functions.

Basics of Linear Algebra

Linear algebra is the mathematics of vectors, matrices, and linear transformations.

It forms the foundation of modern machine learning and data science.

Vectors

A vector is an ordered collection of numbers:

$$x = (x_1, x_2, \dots, x_d) \in \mathbb{R}^d.$$

Vectors represent:

- datapoints,
- embeddings,
- coordinates,
- features.

Example

$$x = (2, 3).$$

This represents a point in two-dimensional space.

Vector Operations

Addition

$$(1, 2) + (3, 4) = (4, 6).$$

Scalar multiplication

$$2(1, 2) = (2, 4).$$

Dot Product

The dot product between two vectors:

$$x \cdot y = \sum_{i=1}^d x_i y_i.$$

Example:

$$(1, 2) \cdot (3, 4) = 1 \cdot 3 + 2 \cdot 4 = 11.$$

The dot product measures similarity and alignment between vectors.

Matrices

A matrix is a rectangular array of numbers:

$$A \in \mathbb{R}^{m \times n}.$$

Matrices represent linear transformations.

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

Matrix-Vector Multiplication

$$Ax$$

maps one vector into another vector.

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Then:

$$Ax = \begin{bmatrix} 3 \\ 7 \end{bmatrix}.$$

Neural networks fundamentally consist of repeated matrix-vector transformations.

Matrix-Matrix Multiplication

Suppose:

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times p}.$$

Their matrix product is:

$$C = AB \in \mathbb{R}^{m \times p}.$$

The element at row i , column j is:

$$C_{ij} = \sum_{k=1}^n A_{ik}B_{kj}.$$

Thus, matrix multiplication combines rows of A with columns of B .

Example

Consider:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

Then:

$$\begin{aligned} AB &= \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix} \\ &= \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}. \end{aligned}$$

Matrix multiplication is fundamental in machine learning because neural networks are essentially repeated matrix multiplications followed by nonlinear functions.

For a neural network layer:

$$y = Wx + b,$$

where:

- x = input vector,
- W = weight matrix,
- b = bias vector,
- y = output vector.

Basics of Optimization

Optimization is the study of finding the best parameters that minimize or maximize an objective function.

In machine learning, optimization is used to learn model parameters from data.

Objective Function

Suppose a model has parameters:

$$\theta.$$

Define a loss function:

$$\mathcal{L}(\theta).$$

The goal is:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta).$$

This means finding parameters that minimize prediction error.

Example

Suppose:

$$\mathcal{L}(x) = x^2.$$

The minimum occurs at:

$$x = 0.$$

Gradient Descent

Gradient descent is the most fundamental optimization algorithm.

Update rule:

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t).$$

where:

- η = learning rate,
- $\nabla \mathcal{L}$ = gradient of the loss.

Interpretation:

- gradient points toward increasing loss,
- negative gradient points toward decreasing loss.

Thus, gradient descent repeatedly moves parameters toward lower loss regions.

Example of Gradient Descent

Let:

$$\mathcal{L}(x) = x^2.$$

Then:

$$\frac{d\mathcal{L}}{dx} = 2x.$$

Gradient descent update:

$$x_{t+1} = x_t - \eta(2x_t).$$

If:

$$x_0 = 4, \quad \eta = 0.1,$$

then:

$$x_1 = 4 - 0.1(8) = 3.2.$$

The parameter gradually moves toward:

$$x = 0,$$

which minimizes the loss.

Optimization in Machine Learning

Training a machine learning model is fundamentally an optimization problem.

The workflow is:

Model $\implies$ Loss Function $\implies$ Optimization $\implies$ Learned Parameters.

Modern deep learning trains millions or billions of parameters using gradient-based optimization methods.

Variants of Gradient Descent

Gradient descent updates parameters using gradients of the loss function.

General update rule:

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t),$$

where:

- θ = model parameters,
- η = learning rate,
- $\nabla \mathcal{L}$ = gradient of loss.

Different optimization algorithms modify this update rule for better convergence and stability.

(i) Batch Gradient Descent

Uses the entire dataset to compute gradients.

Update:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \mathcal{L}_i.$$

Characteristics:

- stable updates,
- computationally expensive,
- slow for very large datasets.

(ii) Stochastic Gradient Descent (SGD)

Instead of using the entire dataset, SGD uses a single sample:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_i.$$

Characteristics:

- much faster updates,
- noisy gradients,
- can escape shallow local minima,
- widely used in deep learning.

Mini-batch SGD is the practical standard:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} \mathcal{L}_i.$$

where B is the batch size.

(iii) RMSProp

RMSProp adapts learning rates individually for each parameter.

It keeps a moving average of squared gradients:

$$s_t = \beta s_{t-1} + (1 - \beta) g_t^2,$$

where:

$$g_t = \nabla_{\theta} \mathcal{L}.$$

Update rule:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t + \epsilon}} g_t.$$

Characteristics:

- stabilizes training,
- prevents extremely large updates,
- useful for non-stationary objectives.

(iv) Adam Optimizer

Adam combines:

- momentum,
- adaptive learning rates.

It maintains:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

(first moment / momentum)
and

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

(second moment / variance estimate).

Bias-corrected estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t},$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Final update:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}.$$

Characteristics:

- fast convergence,
- adaptive per-parameter learning rates,
- default optimizer in many deep learning applications.

(v) AdamW

AdamW modifies Adam by decoupling weight decay from gradient updates.

Standard Adam mixes regularization into gradients, which can produce undesirable behavior.

AdamW performs:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_t.$$

where:

$$\lambda$$

is the weight decay coefficient.

Characteristics:

- better generalization,
- improved stability,
- widely used in transformers and large language models.

Optimization Intuition

All optimization algorithms attempt to minimize a loss surface:

$$\mathcal{L}(\theta).$$

Different optimizers differ in:

- how gradients are estimated,
- how learning rates are adjusted,
- how past gradients influence updates.

Conceptually:

$$\text{Parameters} \implies \text{Loss} \implies \text{Gradient} \implies \text{Parameter Update}.$$

Modern deep learning is fundamentally large-scale optimization in extremely high-dimensional parameter spaces.